The Formulas

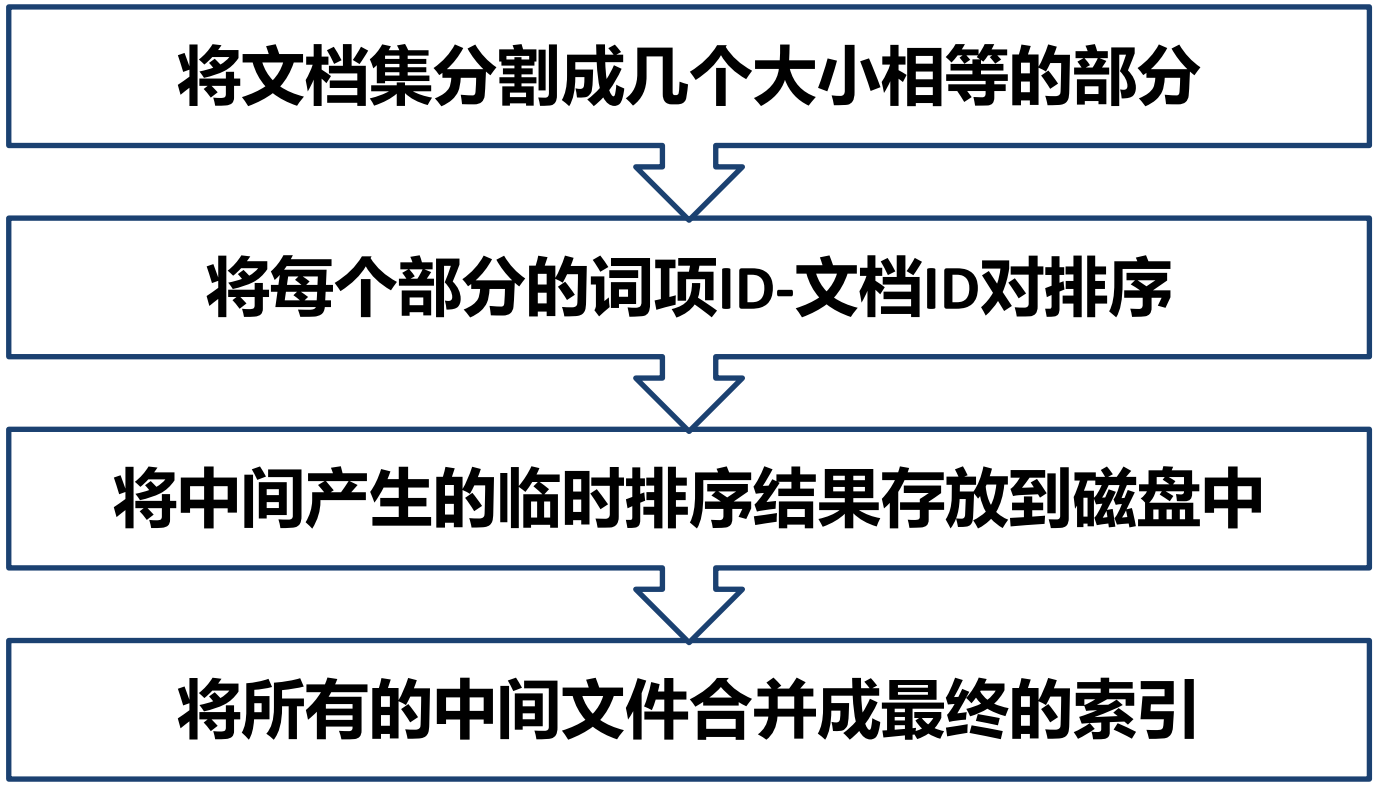
- $1.01^{365}=37.8$
- $1.02^{365}=1377.4$
- $0.99^{365}=0.03$
- $0.98^{365}=0.0006$

Review

- **文档**（document）指的是检索系统的检索对象，它们可以是一条条单独的记录或者是一本书的各章
- 所有的文档组成**文档集**（collection），有时也称为语料库（corpus）
- **词项**（term）是索引的单位，它通常可以用词来表示
- 词条化是将给定的字符序列拆分成一系列子序列的过程，其中每个子序列称为一个**词条**（token）

基于块的排序索引构建算法BSBI
（Blocked Sort-Based Indexing）

算法的基本思路



内存式单遍扫描索引构建算法SPIMI

(Single-pass in-memory indexing)

- **关键思想**
  - **对每个块都产生一个独立的词典**
    - 不需要在块之间进行term-termID的映射
  - **对倒排记录表不排序，按照出现的先后顺序排列**
- 基于上述思想可以对**每个块**生成一个完整的倒排索引
- 这些独立的索引最后合并一个**大索引**

SPIMI算法中块倒排索引的生成

```
SPIMI-INVERT(token_stream)
 1  output_file ← NEWFILE()
 2  dictionary ← NEWHASH()
 3  while (free memory available)
 4  do token ← next(token_stream)
 5     if term(token) ∉ dictionary
 6        then postings_list ← ADDTODICTIONARY(dictionary, term(token))
 7        else postings_list ← GETPOSTINGSLIST(dictionary, term(token))
 8     if full(postings_list)
 9        then postings_list ← DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11  sorted_terms ← SORTTERMS(dictionary)
12  WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13  return output_file
Merging of blocks is analogous to BSBI.
```

- 算法逐一处理（Line 4）每个词项-文档ID对
- 如果词项是出现则直接返回其倒排记录表（Line 7）第一次出现，那么将之加入词典（最好通过哈希表来实现），同时建立一个新的倒排记录表（Line 6）
- 如果该词项不是第一次出现，则直接返回其倒排记录表（Line 7）

BSBI与SPIMI算法比较

- **基于块的排序索引构建算法**BSBI
  - 一开始就整理出所有的**词项ID-文档ID**并对它们进行排序
- **内存式单遍扫描索引构建算法**SPIMI
  - 直接在倒排记录表中增加一项（L10）
  - 倒排记录表**动态增长**
  - 立即就可以实现全体倒排记录表的收集

```
10    AddToPostingsList(postings_list, docID(token))
```

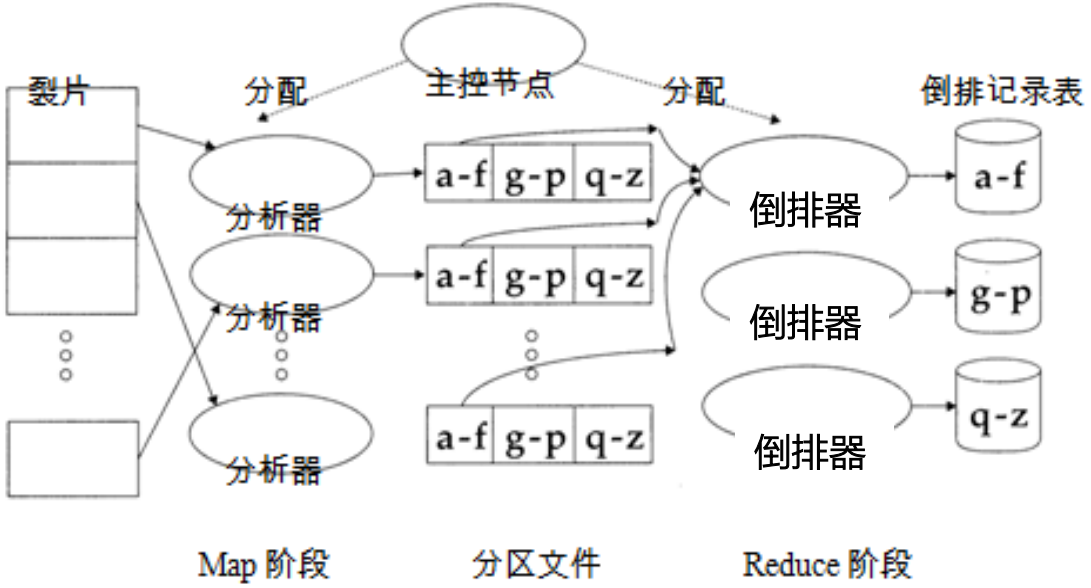
Map-Reduce

- 两类并行任务分配给两类机器：
  - **分析器(Parser)**
  - **倒排器(Inverter)**
- 将输入的文档集**分片(split)** (对应于BSBI/SPIMI算法中的**块**)
- 每个数据片都是一个文档子集

分析器（Parser）

- 思路
  - 主节点将一个数据片分配给一台空闲的分析器
  - **分析器**一次读一篇文档然后输出(term,docID)-对
  - 分析器将这些(term,docID)-对又分成 j 个**词项分区**
  - **每个分区按照词项首字母进行划分**
    - E.g., a-f, g-p, q-z (这里 $j = 3$)

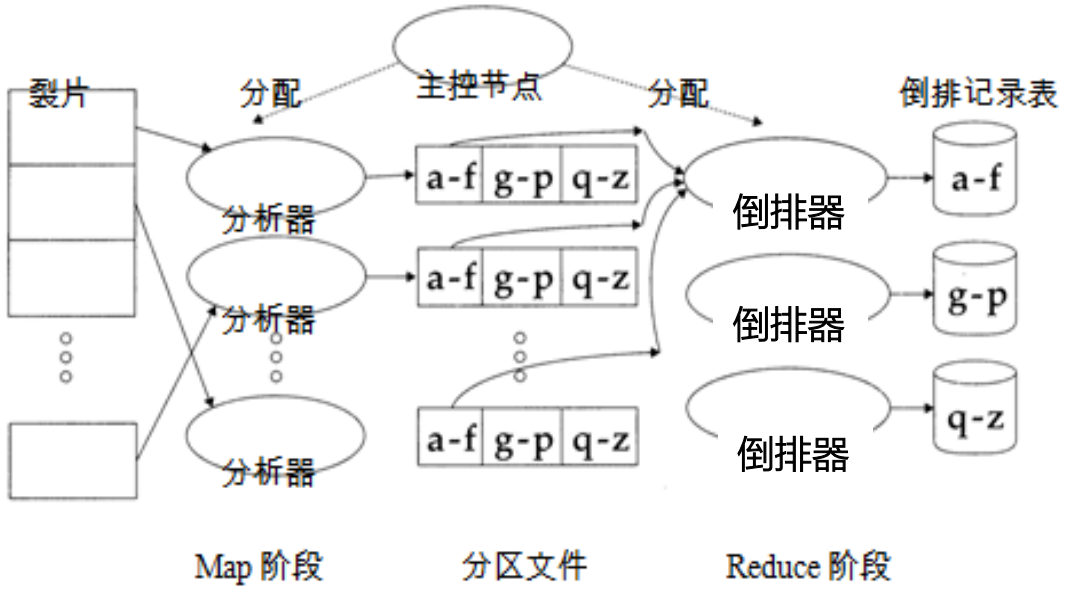
数据流



倒排器 (Inverter)

- 给定一个键（词项ID），将所有的值（文档ID）汇总并组织成倒排表的过程由**Reduce阶段**的**倒排器（inverter）**来完成

数据流



动态索引构建

- 构建
  - 在磁盘上维护一个大的**主索引(Main index)**
  - 新文档放入内存中较小的**辅助索引（Auxiliary index）**中
  - 同时搜索两个索引，然后合并结果
  - **定期将辅助索引合并到主索引中**
- 删除的处理
  - 采用无效位向量(Invalidation bit-vector)来表示删除文档
  - 利用该维向量过滤返回的结果，以去掉已删除文档

对数合并(Logarithmic merge)

- 实际上，采用对数合并方法，信息检索系统的各个方面，包括索引维护、查询处理、分布等等，都要**复杂得多**
- 由于动态索引构建的复杂性，一些大型搜索引擎采用**从头开始重构索引**而不是动态构建索引的方法

1. **周期性地构建一个全新的索引**
2. **将查询处理转到新索引**
3. **同时将旧索引删除**

其他索引

- **以文档ID排序的索引（传统布尔检索）**
  - 新文档的加入往往只需在**倒排记录表最后增加信息**
- **以文档影响度排序**的索引(**排序式检索**)
  - 新文档的加入会导致在**任意可能的地方插入信息**
  - 会大大增加倒排表更新的复杂度

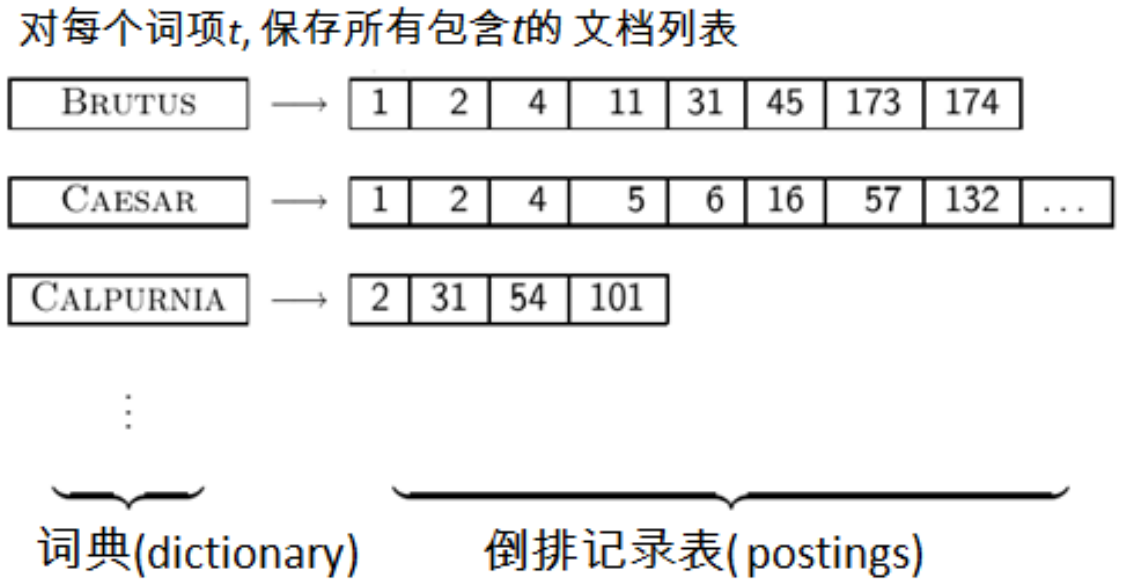
第4讲 索引压缩

Index compression

提纲

1. 上一讲回顾
2. 压缩
3. 词项统计量
4. 词典压缩
5. 倒排记录表压缩

本讲内容



- 信息检索中进行压缩的**动机**
- **词项统计量**: 词项在整个文档集中如何分布?
- 倒排索引中**词典**部分如何压缩?
- 倒排索引中**倒排记录表**部分如何压缩?

5.1 节 信息检索中词项的统计特性

- 在大规模文档集中待压缩对象（包括词项和倒排记录）的统计特性

5.2 节 词典压缩

- “将词典视为长串”
- 按块存储

5.3节 倒排记录表的压缩

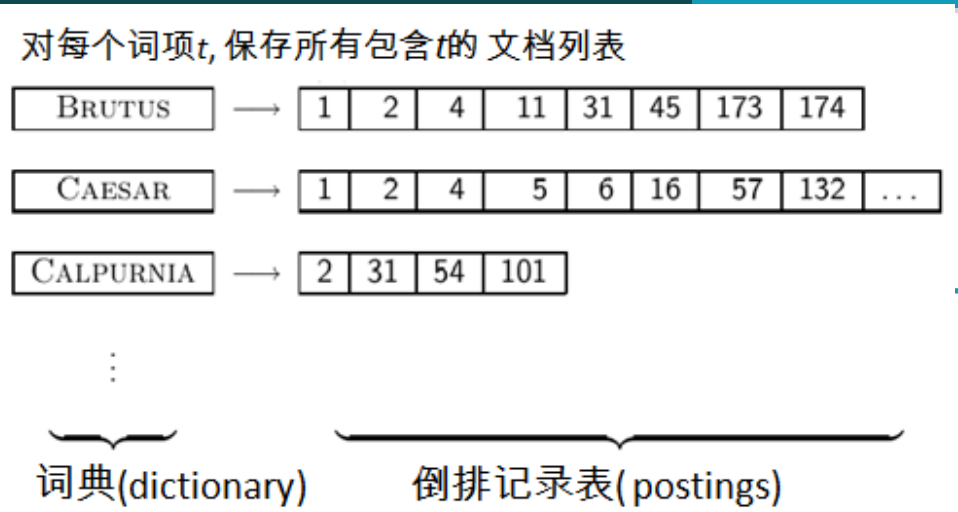
- 变长字节编码
- γ 编码

提纲

1. 上一讲回顾
2. **压缩**
3. 词项统计量
4. 词典压缩
5. 倒排记录表压缩

信息检索系统中的两个主要数据结构

- 词典
- 倒排索引



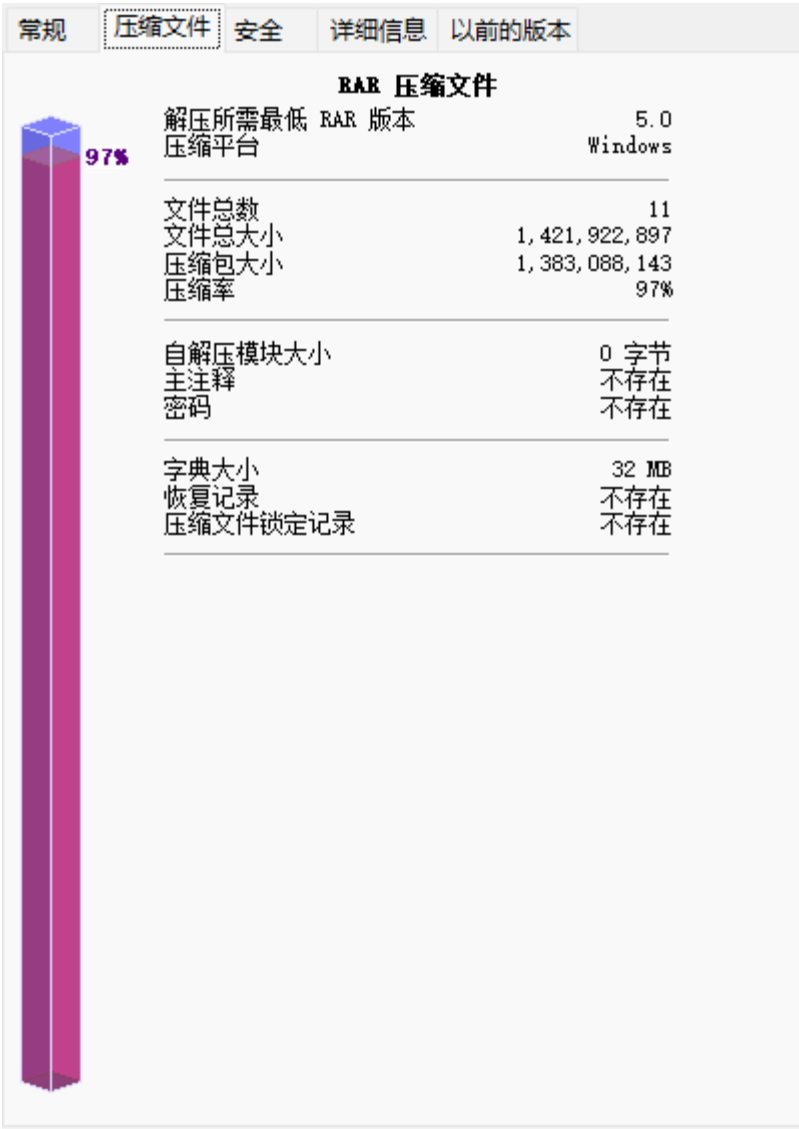
压缩技术

压缩优点

- 节省磁盘空间
- 本章例子：达到1：4的压缩比、降低75%的索引存储开销


2022-2023 业报告 属性
常规
共享
安全
以前的版本
自定义
2022-2023- 业报告
类型: 文件夹
位置: F:\za\2022-2023-
大小: 1.51 GB (1,624,142,660 字节)
占用空间: 1.51 GB (1,624,264,704 字节)
包含: 57 个文件，14 个文件夹
创建时间: 2023年6月28日，15:38:17
属性:
只读(仅应用于文件夹中的文件)(R)
隐藏(H)
高级(D)...
确定
取消
应用(A)


性
常规
压缩文件
安全
详细信息
以前的版本
文件类型: WinRAR 压缩文件 (.rar)
打开方式: WinRAR 压缩文件管理器
更改(C)...
位置: F:\za 课程答辩
大小: 1.28 GB (1,383,090,361 字节)
占用空间: 1.28 GB (1,383,092,224 字节)
创建时间: 2023年6月27日，12:40:30
修改时间: 2023年6月22日，12:06:54
访问时间: 2023年11月15日，16:52:18
属性:
只读(R)
隐藏(H)
高级(D)...
确定
取消
应用(A)


常规
压缩文件
安全
详细信息
以前的版本
RAR 压缩文件
97%
解压所需最低 RAR 版本 5.0
压缩平台 Windows
文件总数 11
文件总大小 1,421,922,897
压缩包大小 1,383,088,143
压缩率 97%
自解压模块大小 0 字节
主注释 不存在
密码 不存在
字典大小 32 MB
恢复记录 不存在
压缩文件锁定记录 不存在

常规 | 共享 | 安全 | 以前的版本 | 自定义

项目整体源码

类型: 文件夹

位置: F:\Downloads

大小: 601 MB (631,219,703 字节)

占用空间: 659 MB (691,101,696 字节)

包含: 43,966 个文件，7,740 个文件夹

创建时间: 2023年11月15日，16:59:41

属性: 只读(仅应用于文件夹中的文件)(R)

隐藏(H) 高级(D)...

项目整体源码

文件类型: WinRAR 压缩文件 (.rar)

打开方式: WinRAR 压缩文件管理器 更改(C)...

位置: F:\Downloads

大小: 212 MB (222,432,358 字节)

占用空间: 212 MB (222,433,280 字节)

创建时间: 2023年11月15日，16:59:35

修改时间: 2022年6月7日，10:04:41

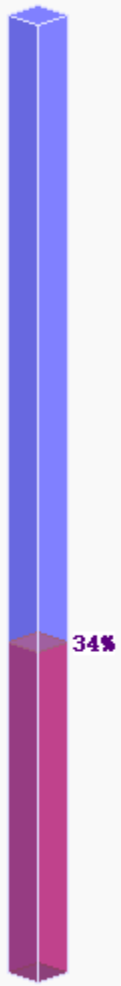
访问时间: 2023年11月15日，16:59:35

属性: 只读(R) 隐藏(H) 高级(D)...

常规 | 压缩文件 | 安全 | 详细信息 | 以前的版本

RAR 压缩文件

解压所需最低 RAR 版本	5.0
压缩平台	Windows
文件总数	43,966
文件总大小	631,219,703
压缩包大小	215,529,954
压缩率	34%
自解压模块大小	0 字节
主注释	不存在
密码	不存在
字典大小	32 MB
恢复记录	不存在
压缩文件锁定记录	不存在



索引压缩隐含优点

1. 能增加高速缓存（caching）技术的利用率

- 在搜索系统中，**词典中某些条目及其索引往往比其他条目及其索引的使用更频繁**
- 例如，如果将一个**频繁使用的查询词项**term的倒排记录表放到**高速缓存**中，那么对仅由term构成的查询进行应答所需要的计算完全可以在**内存**中完成
- **压缩技术，**倒排记录表在内存中解压缩
- 充分减少IR系统的应答时间
- **内存比磁盘更贵**，相对于磁盘**空间**的减少，采用**高速缓存技术带来的速度提升**是采用压缩技术的**更主要**的原因

索引压缩隐含优点

2. 压缩能够加快数据从磁盘到内存的传输速度

- **高效的解压缩算法在现代硬件上运行相当快**
  - **将压缩的数据块传输到内存并解压所需要的总时间往往会比将未压缩的数据块传输到内存要快**
- 举例来说，**即使会增加在内存进行解压缩的开销**，也可以通过加载一个小很多的压缩倒排记录表**减少I/O时间**
- 在大部分情况下，**使用压缩倒排记录表的检索系统会比没用压缩的系统的运行速度要快**

索引压缩隐含优点

2. 压缩能够加快数据从磁盘到内存的传输速度

- 如果压缩的主要目的是为了**节省磁盘空间**，那么压缩算法的**速度**就不用特别考虑
- 但是，**如果要提高高速缓存利用率和磁盘到内存的传输率，则解压缩的速度必须要快**
- 本章介绍的压缩算法都非常高效，都可以达到上面提到的索引压缩的目标
- 本章当中，**倒排记录表中的每条记录定义成文档ID**
  - 例如，**一个倒排记录表为（6; 20, 45,100），其中6是词项ID，它包含3条记录，分别代表不同的文档ID**
- 很多搜索系统中还包括**词频和位置信息**，但是这里我们只考虑简单的**文档ID信息**

什么是**压缩**?

- **111111111111111111**
- **将长编码串用短编码串来代替**
  - **111111111111111111 → 18个1**
  - **18个字节 → 5个字节**

为什么要压缩? (一般意义上而言)

减少磁盘空间(节省开销)

增加内存存储内容

(加快速度、节省时间)

加快从磁盘到内存的数据传输速度(同样加快速度)

- [读压缩数据到内存+在内存中解压]
- 比直接读入未压缩数据要快很多
- 前提: 解压速度很快

为什么在**信息检索IR**中需要压缩?

词典的存储空间

- 词典压缩的主要动机：使之能够尽量放入内存中

倒排记录表

- 动机：减少磁盘存储空间，减少从磁盘读入内存的时间
  - 注意：大型搜索引擎将相当比例的倒排记录表都放入内存

多种机制

- 词典压缩
- 倒排记录表压缩

有损(Lossy) vs. 无损(Lossless)压缩

有损压缩

- 丢弃一些信息
- 前面讲到的很多常用的预处理步骤可以看成是有损压缩
  - 统一小写、去除停用词、Porter词干还原、去掉数字
- 有损音乐格式：MP3、WMA

无损压缩

- 所有信息都保留
- 索引压缩中通常都使用无损压缩
- 无损音乐格式：APE、FLAC、WAV

- 当损失的信息不会被检索系统所用时，有损压缩很有意义
  - 例如，Web搜索
  - 其主要特点是文档数目巨大、查询短、用户只关注头几页结果等等
- 因此可以将那些排名不可能靠前的文档的倒排记录给去掉
- 所以，在某些检索场景下可以使用有损压缩技术，而不会对检索效果造成任何损失

提纲

1. 上一讲回顾
2. 压缩
3. **词项统计量**
4. 词典压缩
5. 倒排记录表压缩

词典压缩和倒排记录表压缩

词典压缩中词典的大小

即词汇表的大小是关键

- 能否预测词典的大小?

倒排记录表压缩中

词项的分布情况是关键

- 能否对词项的分布进行估计?

- 引入词项统计量进行估计
- 两个经验法则

对文档集建模： Reuters RCV1语料库

N	**文档数目**	**800,000**
L	**每篇文档的词条数目**	**200**
M	**词项数目(= 词类数目)**	**400,000**
	每个词条的字节数 (含空格和标点)	**6**
	每个词条的字节数 (不含空格和标点)	**4.5**
	每个词项的字节数	**7.5**
T	**无位置信息索引中的倒排记录数目**	**100,000,000**

索引构建过程: 词条序列

Doc 1

I did enact Julius Caesar I was killed i' the Capitol; Brutus killed me.

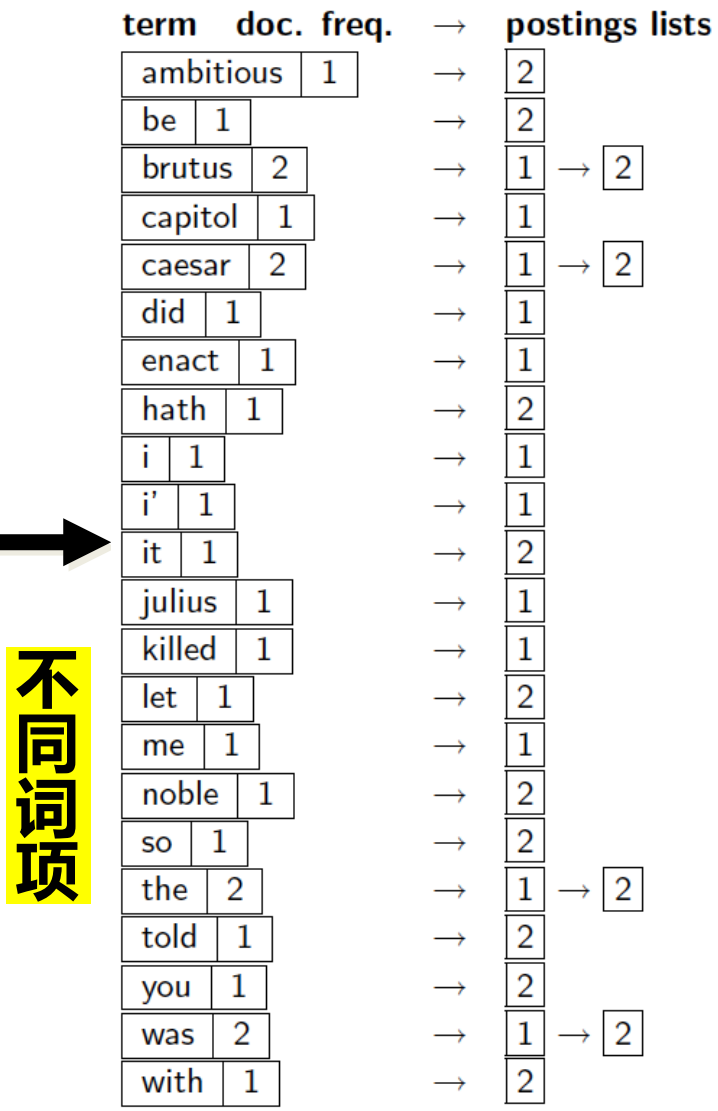
Doc 2

So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious

词条

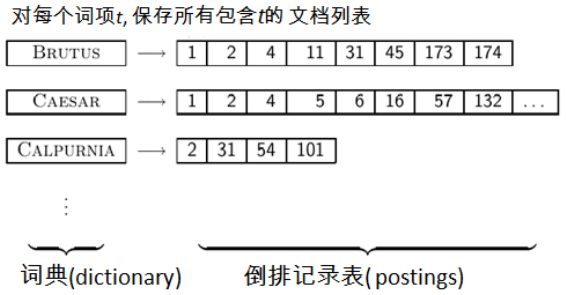
Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

不包含位置信息的索引



不同词项

预处理的效果



	不同词项			无位置信息倒排记录			词　条①		
	数目	Δ%	T%	数目	Δ%	T%	数目	Δ%	T%
未过滤	484 494			109 971 179			197 879 290		
无数字	473 723	–2	–2	100 680 242	–8	–8	179 158 204	–9	–9
大小写转换	391 523	–17	–19	96 969 056	–3	–12	179 158 204	–0	–9
30个停用词	391 493	–0	–19	83 390 443	–14	–24	121 857 825	–31	–38
150个停用词	391 373	–0	–19	67 001 847	–30	–39	94 516 599	–47	–52
词干还原	322 383	–17	–33	63 812 300	–4	–42	94 516 599	–0	–52

Δ%：表示和前一行相比数目的减少比率

T%：表示以未过滤情况为基准的词项数目的累积减少比率

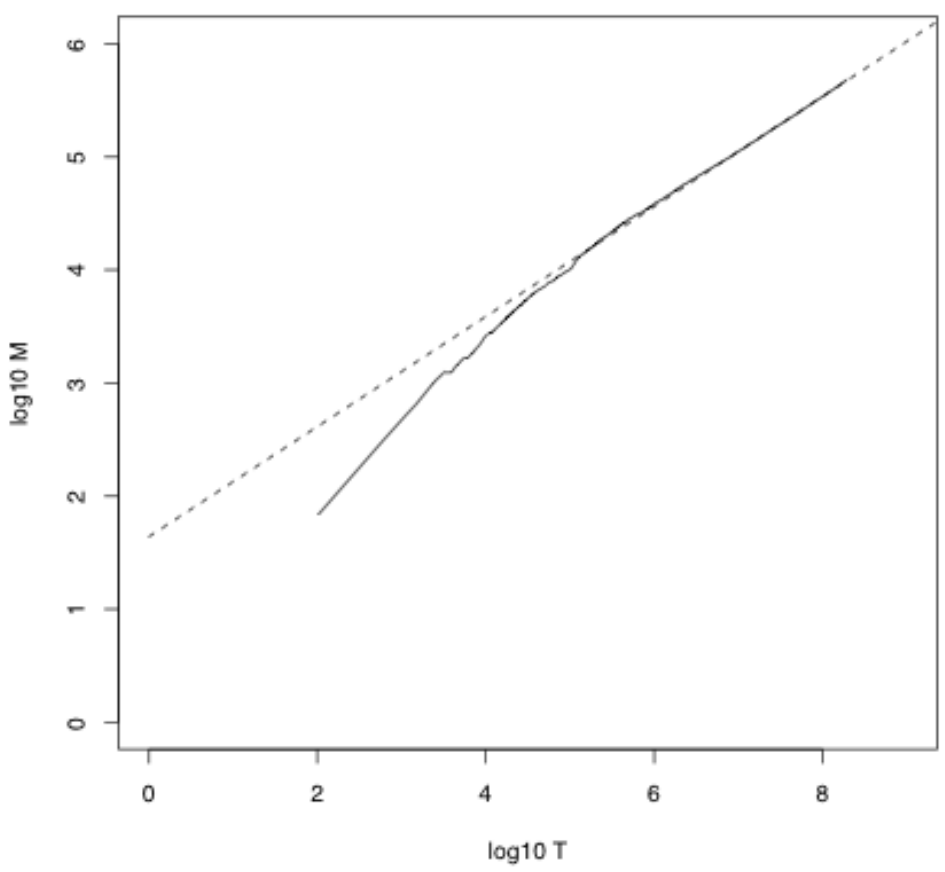
第一个问题：词汇表有多大(即**词项数目**)?

- **即有多少不同的单词数目?**
  - **能否假设这个数目存在一个上界?**
  - **对于长度为20的英文单词，有大约26^{20}种可能的单词**
  - **negative**
- **实际上，词汇表大小会随着文档集的大小增长而增长!**

第一个问题：词汇表有多大(即**词项数目**)?

- 实际上，**词汇表大小会随着文档集的大小增长而增长！**
- **Heaps定律: $M = kT^b$**
- **M 是词汇表大小**
- ***T* 是文档集的大小(所有词条的个数，即所有文档大小之和)**
- 参数k 和b 的一个经典取值是: $30 \leq k \leq 100$ 及 $b \approx 0.5$
- Heaps定律在**对数空间下是线性的**
  - 对数空间下两者之间最简单的关系
  - 经验规律

Reuters RCV1上的Heaps定律



- 词汇表大小M是文档集规模T的一个函数
- 图中通过最小二乘法拟合出的直线方程为：

$$\log_{10} M = 0.49 * \log_{10} T + 1.64$$

- 于是有：
- $M = kT^b = 10^{1.64} T^{0.49}$
- $k = 10^{1.64} \approx 44$
- $b = 0.49$

拟合 vs. 真实

- 例子: 对于RCV1前1,000,020个词条， 根据Heaps定律预计将有38,323个词项:
- $M = kT^b$

$$44 \times 1{,}000{,}020^{0.49} \approx 38{,}323$$

- 实际的词项数目为38,365，和预测值非常接近
- 经验上的观察结果表明，一般情况下拟合度还是非常高的

词汇量的增长率

- 不同的文档集下参数k的取值差异会比较大
- 词汇量的大小依赖于文档集本身以及对它进行处理的方法

降低词汇量的增长率

- 大小写转换
- 词干还原

增加词汇量的增长率

- 允许加入数字
- 容忍拼写错误

两点假设

不论在某个特定的文档集中的参数取值如何，Heaps定律提出了如下两点假设：

I. 随着文档数目的增加，词汇量会持续增长而不会稳定到一个最大值

II. 大规模文档集的词汇量也会非常大

上述两个假设在经验上被认为是正确的，并且已经在很多大规模文档集上都得到了验证

练习

问：在容许拼写错误或者对拼写错误自动纠错的情况下，Heaps 定律的效果如何?

拼写错误举例：mistake、mistakee、mestake

容许拼写错误的词项数量：3

拼写错误自动纠错的词项数量：1

容许拼写错误会增加词汇量的增长率!

练习

计算词汇表大小 M

- **观察一个网页集合，你会发现在前10 000个词条中有3 000个不同的词项，在前1 000 000个词条中有30 000个不同的词项**
- **假定某搜索引擎索引了总共20,000,000,000 (2×10^{10})个网页, 平均每个网页包含200个词条**
- **那么按照Heaps定律，被索引的文档集的词汇表大小是多少?**

课堂练习

解：按照 $M = kT^b$,

$3000 = k \times 10000^b = k \times 10^{4b}$

$30000 = k \times 1000000^b = k \times 10^{6b}$

求出$k = 30, b = 0.5$，**代入公式**$M = kT^b$,

$M = k \times (4 \times 10^{12})^b = 30 \times (4 \times 10^{12})^{0.5} = 6 \times 10^7\ (60M)$

第二个问题：词项的分布如何？Zipf 定律

- **Heaps定律告诉我们随着文档集规模的增长词项的增长情况**
- **但是我们还需要知道在文档集中有多少高频词项 vs. 低频词项**

- **在自然语言中，有一些极高频词项，有大量极低频的罕见词项**
- **Zipf 定律: 第i 常见的词项的频率cf_i和$1/i$ 成正比:** $$cf_i \propto \frac{1}{i}$$
- **cf_i 是文档集频率(collection frequency): 词项t_i在所有文档中出现的次数(不是出现该词项的文档数目df)**

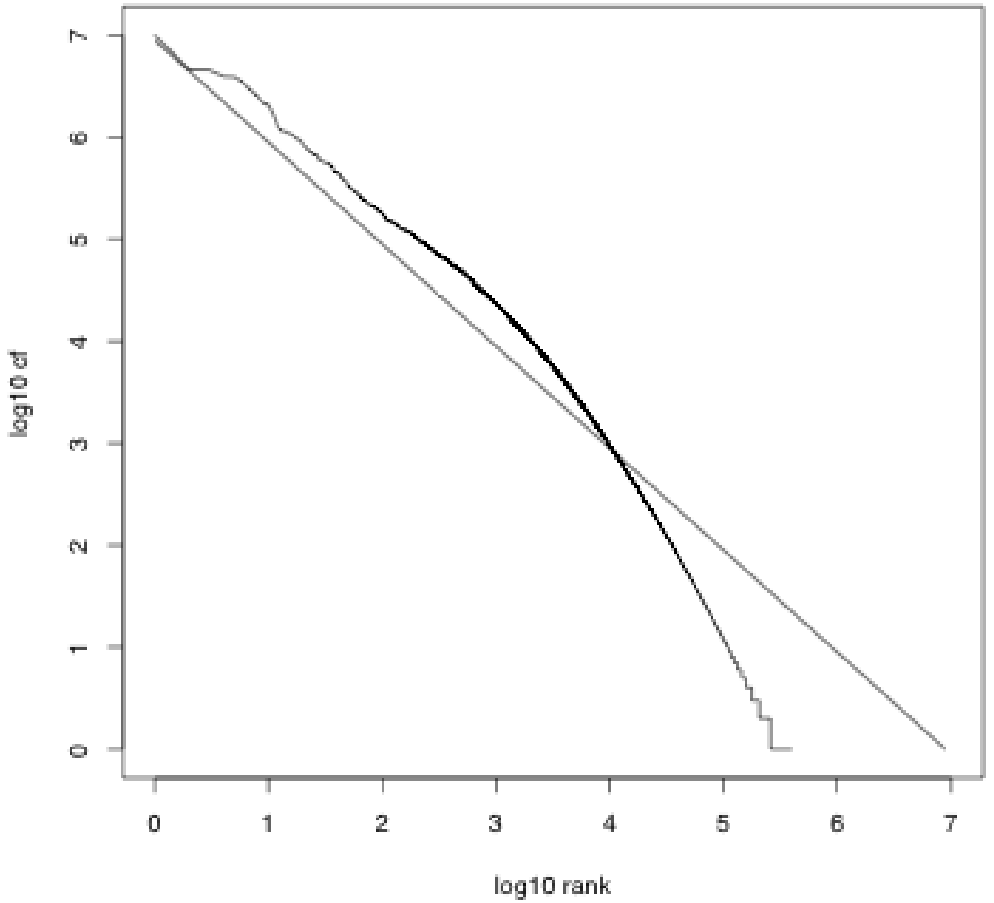
Zipf 定律

- 于是，如果最常见的词项(*the*)出现cf_1 次
- 那么第二常见的词项 (*of*)出现次数为 $cf_2 = \frac{1}{2} cf_1$
- 第三常见的词项 (*and*) 出现次数为 $cf_3 = \frac{1}{3} cf_1$
- 另一种表示方式: $cf_i = ci^k$ 或 $\log cf_i = \log c + k \log i \ \ (k = -1)$

Zipf定律

- 齐夫定律（英语：Zipf's law，IPA/ˈzɪf/）是由哈佛大学的语言学家乔治•金斯利•齐夫（George Kingsley Zipf）于1949年发表的实验定律
- 它可以表述为：在自然语言的语料库里，一个单词出现的频率与它在频率表的排名成反比
  - 频率最高的单词出现的频率大约是出现频率第二位的单词的2倍
  - 出现频率第二位的单词则是出现频率第四位的单词的2倍
- 这个定律被作为任何与幂定律概率分布有关的事物的参考

Reuters RCV1上Zipf定律的体现



词项的出现频率是其排名的函数

- **拟合度不是非常高，但是最重要的是如下关键性发现**
  - **高频词项很少**
  - **低频罕见词项很多**

Q&A

1. 为什么要压缩
2. 索引压缩优点
3. Heaps定律、Zipf定律

提纲

1. 上一讲回顾
2. 压缩
3. 词项统计量
4. **词典压缩**
5. 倒排记录表压缩

词典压缩

- 本节将给出**词典的一系列数据结构**，它们能够达到越来越高的压缩比
- 和**倒排记录表的大小相比，词典只占非常小的空间**

	不同词项			无位置信息倒排记录			词　　条①		
	数目	Δ%	T%	数目	Δ%	T%	数目	Δ%	T%
未过滤	484 494			109 971 179			197 879 290		
无数字	473 723	–2	–2	100 680 242	–8	–8	179 158 204	–9	–9
大小写转换	391 523	–17	–19	96 969 056	–3	–12	179 158 204	–0	–9
30个停用词	391 493	–0	–19	83 390 443	–14	–24	121 857 825	–31	–38
150个停用词	391 373	–0	–19	67 001 847	–30	–39	94 516 599	–47	–52
词干还原	322 383	–17	–33	63 812 300	–4	–42	94 516 599	–0	–52

词典压缩

- **决定信息检索系统查询响应时间的一个重要因素是磁盘的访问次数**
- 而如果有**部分词典存在于磁盘**上，那么在处理查询时就需要更多的磁盘访问次数
- **词典压缩的主要目的是将词典放入内存**，或者说至少要把大部分词典放入内存，这样才能获得很高的查询吞吐率
- 尽管即使是非常**大规模的文档集的词典**也往往能够放入一台标准台式计算机的内存，但是在很多其他场景下情况并非如此

词典压缩

- **例如**
  - 大公司的一台企业搜索服务器也许要索引数太字节的文档，由于文档中可能包含多种不同语言，所以最后的词汇量可能会很大
- 另外，有些搜索系统的设计要考虑到**硬件条件的限制**
  - 手机或者车载计算机上的搜索系统等
- 压缩词典节省内存的其他原因还包括**快速启动**以及**与其他应用程序共享资源**等
  - 运行搜索系统的PC上可能同时也在运行作为内存占用“大户”的字处理软件

词典压缩

- 动机总结
  - 相对于倒排记录表，词项较小
  - 但是我们想将词典放入内存
  - 另外，满足一些特定领域特定应用的需要，如手机、机载计算机上的应用或要求快速启动等需求
- 压缩词典相当重要

5.1 节 信息检索中词项的统计特性

- 在大规模文档集中待压缩对象（包括词项和倒排记录）的统计特性

5.2 节 词典压缩

- “将词典视为长串”
- 按块存储

5.3节 倒排记录表的压缩

- 变长字节编码
- γ 编码

回顾: 定长数组方式下的词典存储

- 每个**词项**采用20B(Byte)的固定长度（英文中很少有长度大于20B的词）存储
- **文档频率**采用4B存储
- 词项到倒排记录表的**地址指针**也采用4B存储
  - **4B的指针能够访问4GB的地址空间**

词项	文档频率	指向倒排记录表的指针
a	656 265	→
aachen	65	→
…	…	…
zulu	221	→

空间需求：　　20 字节　　4 字节　　4 字节

对文档集建模： Reuters RCV1语料库

N	文档数目	800,000
L	每篇文档的词条数目	200
M	词项数目(= 词类数目)	400,000
	每个词条的字节数 (含空格和标点)	6
	每个词条的字节数 (不含空格和标点)	4.5
	每个词项的字节数	7.5
T	无位置信息索引中的倒排记录数目	100,000,000

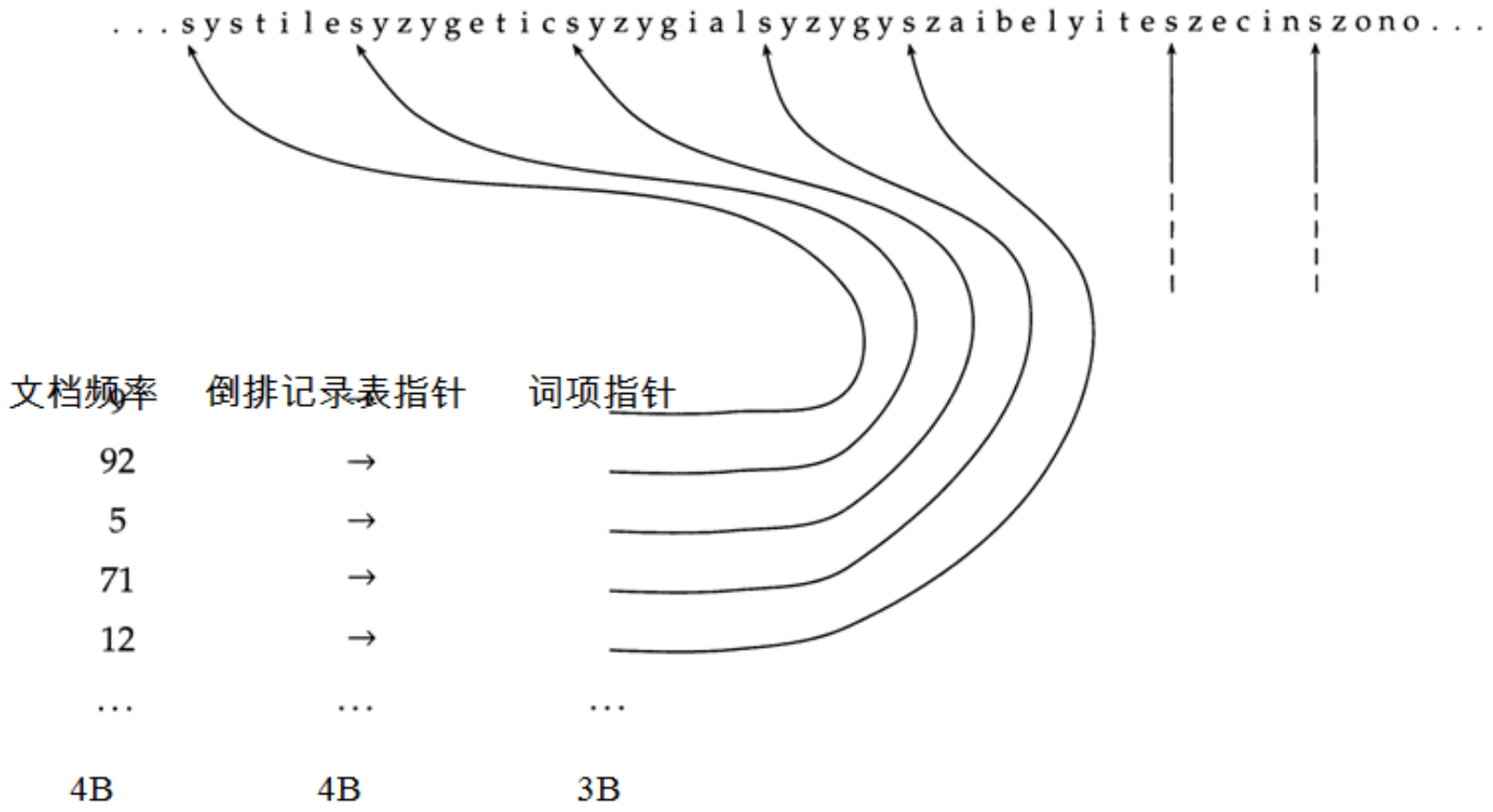
Reuters RCV1语料库的存储空间?

$$(20+4+4)\text{B} \times 400{,}000 = 11.2\text{MB}$$

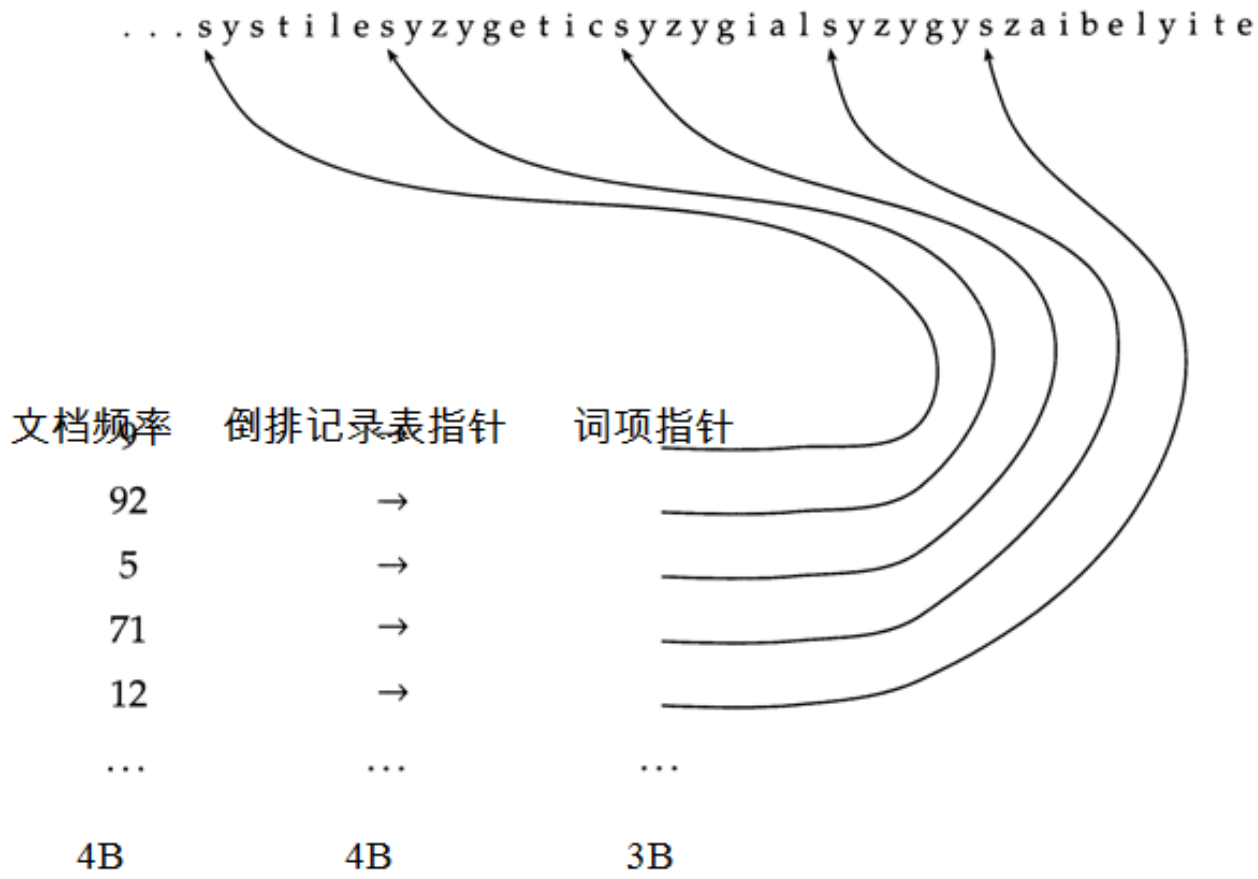
定长方式的不足

- **大量存储空间被浪费**
  - 即使是**长度为1的词项，我们也分配20个字节**
- **不能处理长度大于20字节的词项**，如
  - ✓ HYDROCHLOROFLUOROCARBONS (HCFCS)
  - ✓ SUPERCALIFRAGILISTICEXPIALIDOCIOUS
- 而**英语中每个词项的平均长度为8个字符**
- **能否对每个词项平均只使用8个字节来存储?**

1.将整部词典看成单一字符串
(Dictionary as a string)



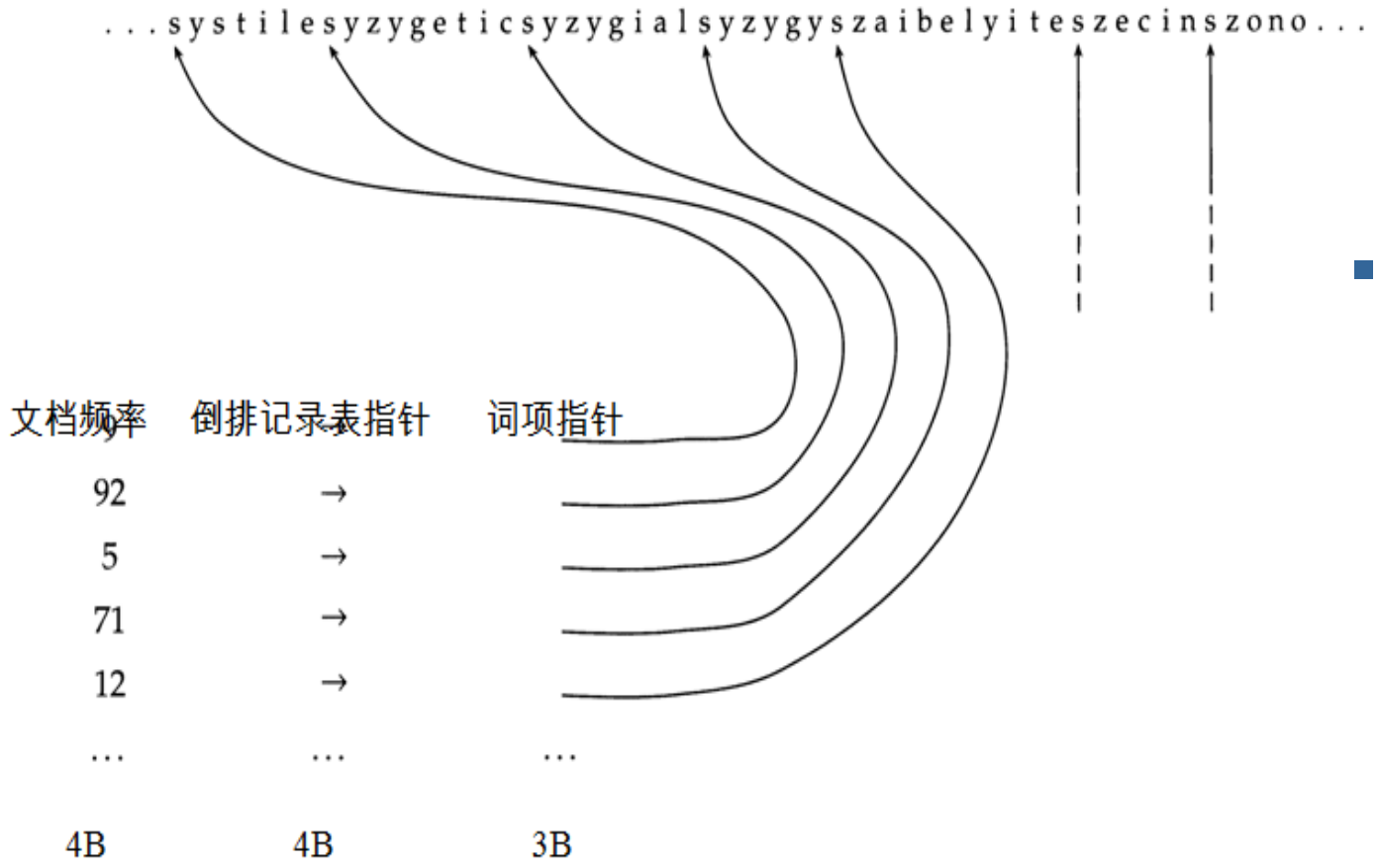
将整部词典看成单一字符串(Dictionary as a string)



单一字符串方式下的空间消耗

- **每个词项的文档频率需要4个字节**
- **每个词项指向倒排记录表的指针需要4个字节**
- **每个词项平均需要8个字节**

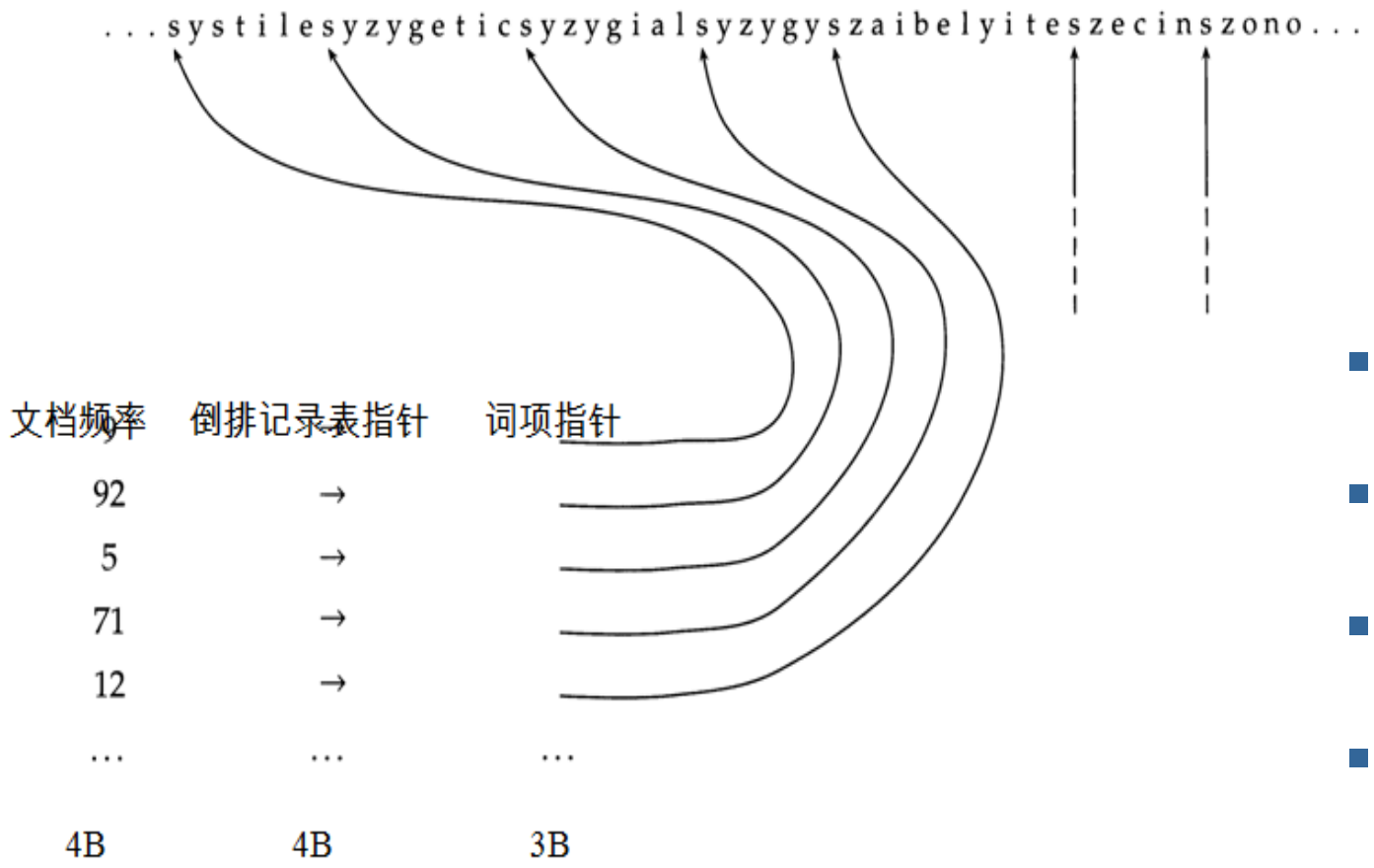
将整部词典看成单一字符串(Dictionary as a string)



单一字符串方式下的空间消耗

- **指向字符串的指针需要3个字节**
  - **指针寻址空间大小(8*400000)个位置**
  - **一个指针需要log2（8 * 400000） < 24 位来表示)**

将整部词典看成单一字符串(Dictionary as a string)



单一字符串方式下的空间消耗

- **每个词项的平均长度是8B**
- **空间消耗**
- **400,000 × (4+4+3+8) = 7.6MB**
- **(定长数组方式需要11.2MB)**

将整部词典看成单一字符串(Dictionary as a string)

	定长数组方式下的词典存储			单一字符串方式下的空间消耗			
	词项长度	文档频率	倒排指针	词项长度	文档频率	倒排指针	词项指针
空间需求	20	4	4	8	4	4	3
	(20 + 4 + 4) B ×400,000 = 11.2 MB			(8 + 4 + 4 + 3) B × 400,000 = 7.6 MB			

2.单一字符串方式下按块存储

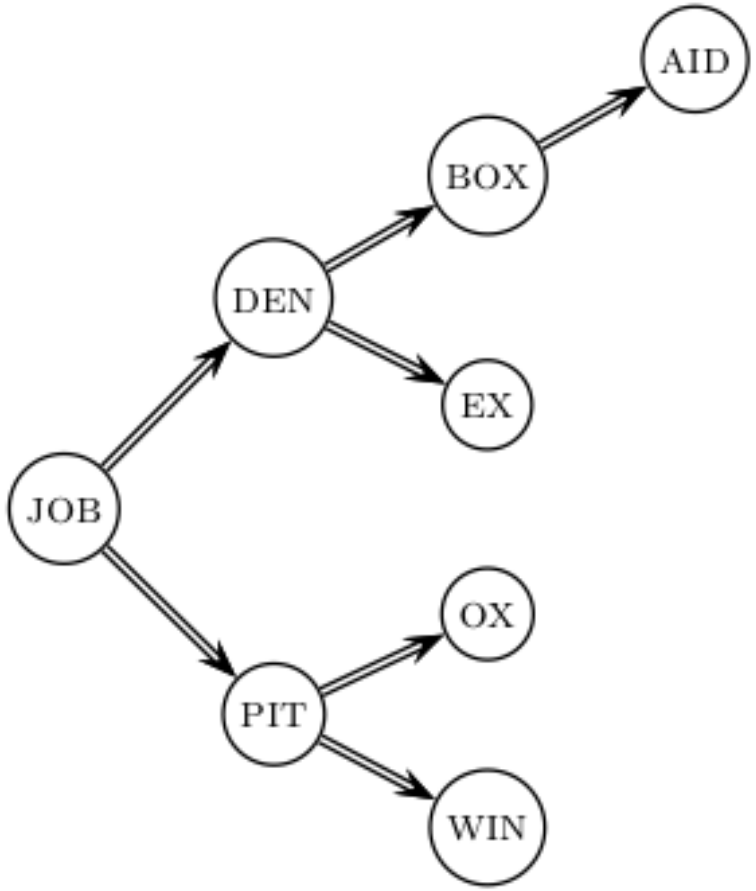


按块存储下的空间消耗

- 如果不按块存储，则每4个**词项指针**将占据空间4 × 3=12B
- 现在按块存储，假设**块大小k**=4，此时**每4个词项只需要保留1个词项指针**，但是**同时需要增加4个字节来表示每个词项的长度**，此时**每4个词项需要3+4=7B**
- 因此，每4个词项将节省12-7=5B
- 于是，整个词典空间将节省400,000/4*5B=0.5MB
- 最终的词典空间将从7.6MB压缩至7.1MB

不采用块存储方式下的词项查找

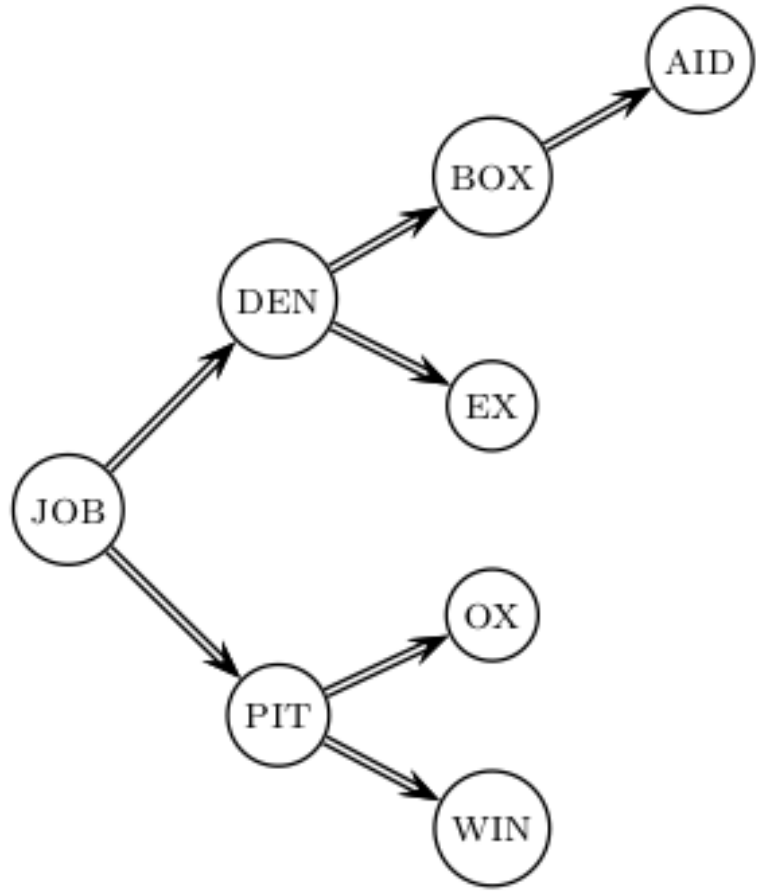
- **显然，块大小k越大，压缩率也越高**
- **But 在压缩和词项查找速度之间必须要保持某种平衡**



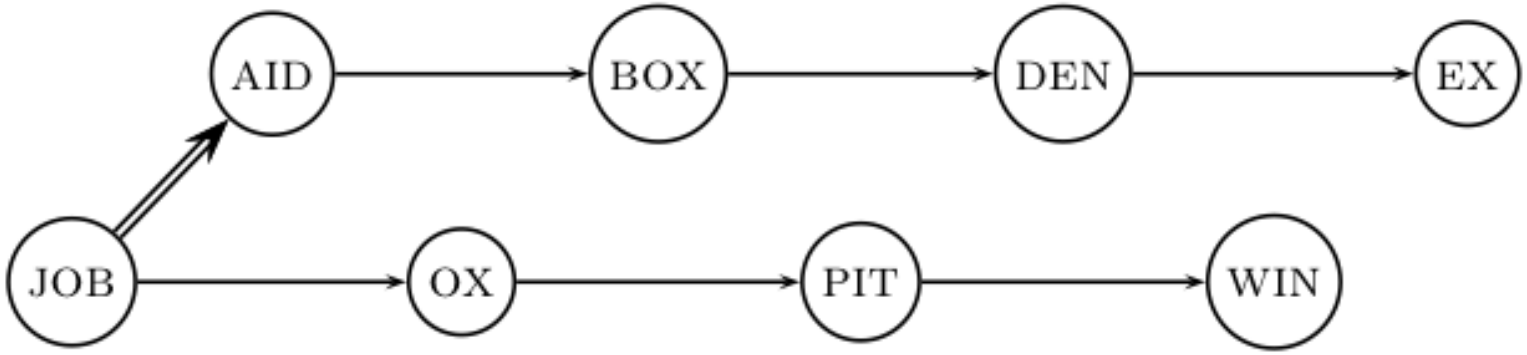
- 8个词项组成的词典(右图)
- 采用**二分法搜索**的步骤在图中以双线表示
- 采用**表搜索（list search）**的步骤则以单线表示

不采用块存储方式下的词项查找

- 当词典**未压缩**时，可以采用**二分查找**方法搜索词项
- **假定每个后续词项的出现概率都相等**的话，那么在未压缩的词典中**查找的平均时间**为(0 + 1 + 2 + 3 + 2 + 1 + 2 + 2) / 8≈ 1.6 步
- 比如，查找aid及box两个词项，就分别需要3步和2步

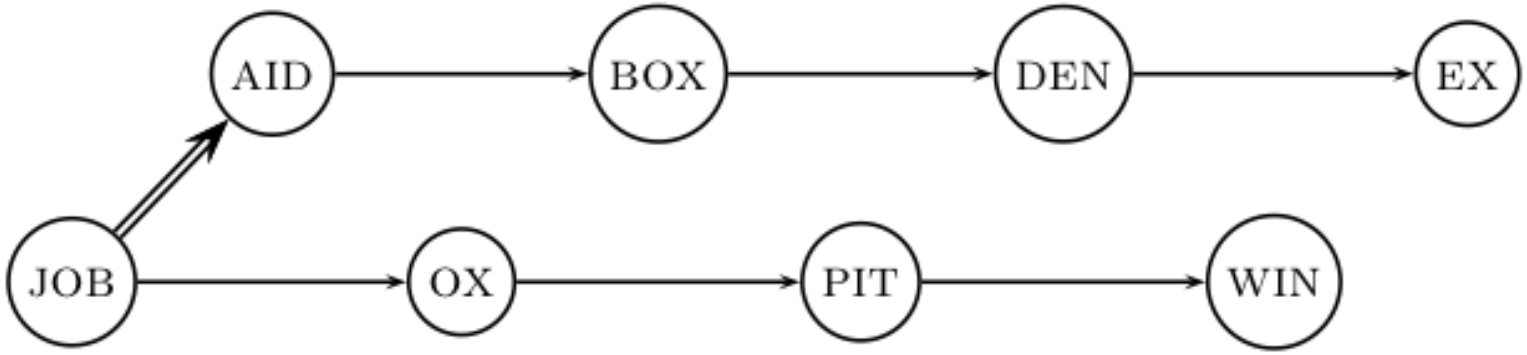


采用块存储方式下的词项查找: 稍慢



- 在**压缩后**的词典中，首先通过**二分查找**得到块的入口位置，然后在块内进行**线性查找**得到最后的词项位置
- 如果块的大小为4，则平均查找时间为(0 + 1 + 2 + 3 + 4 + 1 + 2 + 3) ／8 = 2 步，和前面相比，多花费了近25%的时间

采用块存储方式下的词项查找: 稍慢



- 比如，寻找den需要1步二分搜索加上2步块内搜索。通过增加k，可以把词典压缩到无限趋近于最小值
- 400 000 × (4+ 4 + 1 + 8) = 6.8 MB
- 此时词典的查找会因为k很大而慢得不可忍受

单一字符串方式下的空间消耗			
词项长度	文档频率	倒排指针	词项指针
8	4	4	3
(8 + 4 + 4 + 3) B × 400,000 = 7.6 MB			

前端编码(Front coding)

- 按照词典顺序排序的连续词项之间往往具有公共前缀
- **前端编码（front coding）**
- 公共前缀被识别出来之后，后续的词项中便可以使用一个特殊字符来表示这段前缀
- 每个块当中 (k = 4)，会有公共前缀

前端编码(Front coding)

- 每个块当中 (k = 4)，会有公共前缀
- 8automata8automate9automatic10automation
- ⇓
- 可以采用前端编码方式继续压缩
- 8automat*a1◊e2◊ic3◊ion

- 多个连续词项具有公共前缀
- automat
- 在前缀的末尾用*号标识
- 后续的词项中用◊表示该前缀
- 每个词项的前面第一个字节存储了该词项的长度

Reuters RCV1**词典压缩**情况总表

数据结构	空间大小（单位：MB）
词典，定长数组	11.2
词典，长字符串+词项指针	7.6
词典，按块存储，k=4	7.1
词典，按块存储+前端编码	5.9

其他压缩方法

- **最小完美哈希（minimal perfect hashing）**
  - 具有更高压缩率
  - 该哈希函数将M个词项映射到[1,…,M]上，并且不会发生任何冲突
  - 但是，当插入新词项时，显然会发生冲突，此时不能对原有的完美哈希结果进行增量式修改而只能重新构造新的完美哈希函数
  - 无法在动态环境下使用
- **B-树**
  - 即使采用最好的压缩方法，在很多情况下（比如文档集规模很大而内存很小时）要将所有词典存入内存也是不可能的
  - 如果不得不把词典划分成不同页存储在磁盘上，则可以采用B-树对**每页的第一个词项进行索引**
  - 在处理大多数查询时，搜索系统必须要到磁盘获取倒排记录表，而在分页方式下还需要首先从磁盘获取词项所在的词典页，这会显著增加查询处理的时间，不过这种增加是可以忍受的

提纲

1. 上一讲回顾
2. 压缩
3. 词项统计量
4. 词典压缩
5. **倒排记录表压缩**

倒排记录表压缩

- **倒排记录表空间远大于词典，至少10倍以上**
- **压缩关键：对每条倒排记录进行压缩**
- **目前每条倒排记录表中存放的是docID**



	不同词项			无位置信息倒排记录			词　　条①		
	数目	Δ%	T%	数目	Δ%	T%	数目	Δ%	T%
未过滤	484 494			109 971 179			197 879 290		
无数字	473 723	–2	–2	100 680 242	–8	–8	179 158 204	–9	–9
大小写转换	391 523	–17	–19	96 969 056	–3	–12	179 158 204	–0	–9
30个停用词	391 493	–0	–19	83 390 443	–14	–24	121 857 825	–31	–38
150个停用词	391 373	–0	–19	67 001 847	–30	–39	94 516 599	–47	–52
词干还原	322 383	–17	–33	63 812 300	–4	–42	94 516 599	–0	–52

倒排记录表压缩

- 对于Reuters RCV1(800,000篇文档), 每个docID可以采用4字节（即32位）整数来表示
- 当然，我们也可以采用$\log_2 800{,}000 \approx 19.6 < 20$位来表示每个docID
- 我们的压缩目标是
  - 压缩后每个docID用到的位数远小于20比特 (WHY?)

关键思想: 存储docID间隔而不是docID本身

- **每个倒排记录表中的docID是从低到高排序**
  - **例子: COMPUTER: 283154, 283159, 283202, . . .**

- **存储间隔能够降低开销**
- **283159-283154 = 5**
- **283202-283159 = 43**

关键思想: 存储docID间隔而不是docID本身

- **每个倒排记录表中的docID是从低到高排序**
  - **例子: COMPUTER: 283154, 283159, 283202, . . .**

- **于是可以顺序存储间隔(第一个值不是间隔)**
- **COMPUTER: 283154, 5, 43, . . .**
- **高频词项的间隔较小**
- **因此，可以对这些间隔采用小于20比特的存储方式**

对间隔编码

	编码对象	倒排记录表					
the	文档ID	…	283 042	283 043	283 044	283 045	…
	文档ID间距			1	1	1	…
computer	文档ID	…	283 047	283 154	283 159	283 202	…
	文档ID间距			107	5	43	…
arachnocentric	文档ID	252 000	500 100				
	文档ID间距	252 000	248 100				

变长编码

- **目标**
  - **ARACHNOCENTRIC 及其他罕见词项**
    - **每个间隔仍然使用20比特**
    - **Arachnocentric: 心律不齐**
  - **THE及其他高频词项**
    - **每个间隔仅仅使用很少的比特位来编码**

变长编码

- **为了实现上述目标，需要设计一个变长编码**
- **variable length encoding**
- **可变长编码**
  - **对于短间隔采用短编码**
  - **对于长间隔采用长编码**

变长编码

- **对小数字采用比大数字更短的编码方式**
- **间距 GAP**
  - **按字节压缩**
  - **按位压缩**

可变字节(Variable Byte, VB)码

- VB（Variable byte, 可变字节）编码
  - 利用整数个字节来对间距编码
- 被很多商用/研究系统所采用
- 变长编码及对齐敏感性(指匹配时按字节对齐还是按照位对齐)的简单且不错的混合产物

可变字节(Variable Byte, VB)码

- VB（Variable byte, 可变字节）编码
  - 利用整数个字节来对间距编码
  - 后7位是间距的有效编码区
  - 第1位是延续位（continuation bit）
  - 如果该位为1，则表明本字节是某个间距编码的最后一个字节，否则不是

可变字节(Variable Byte, VB)码

- 设定一个专用位 (高位) c 作为延续位(continuation bit)
- 如果间隔表示少于7比特，那么c 置1，将间隔编入一个字节的后7位中
- 否则：将低7位放入当前字节中，并将c 置0，剩下的位数采用同样的方法进行处理
- 最后一个字节的c 置1（表示结束）

VB 编码的例子

文档ID	824	829	215 406
间距		5	214 577
VB编码	00000110 10111000	10000101	00001101 00001100 10110001

VB 编码和解码过程

```
VBEncodeNumber(n)
1  bytes ← ⟨⟩
2  while true
3  do Prepend(bytes, n mod 128)
4     if n < 128
5        then Break
6     n ← n div 128
7  bytes[Length(bytes)] += 128
8  return bytes
```

```
VBEncode(numbers)
1  bytestream ← ⟨⟩
2  for each n ∈ numbers
3  do bytes ← VBEncodeNumber(n)
4     bytestream ← Extend(bytestream, bytes)
5  return bytestream
```

```
VBDecode(bytestream)
1  numbers ← ⟨⟩
2  n ← 0
3  for i ← 1 to Length(bytestream)
4  do if bytestream[i] < 128
5        then n ← 128 × n + bytestream[i]
6        else n ← 128 × n + (bytestream[i] − 128)
7             Append(numbers, n)
8             n ← 0
9  return numbers
```

div和mod函数分别计算整数相除和相除后的余数

其它编码

- 除字节外，还可以采用不同的对齐单位
  - 比如32位(word)、16位及4位(nibble)等等
- 如果有很多很小的间隔，那么采用可变字节码会浪费很多空间，而此时采用4位为单位将会节省空间
- 编码单位越长，那么所需的位操作次数也越少，但是同时压缩率会降低甚至没有压缩
- 更短的编码单位会得到更高的压缩率，但同时位操作的次数也越多
- 最近一些工作采用了32位的方式

γ编码

- 另外一种变长编码是基于位的编码
  - γ编码是其中最出名的一种
- VB 编码根据间距的大小采用合适的字节数来编码
- 基于位的编码能够在更细的位粒度上进行编码长度的自适应调整
- 最简单的位编码是一元编码（unary code）

γ编码

- **一元码(unary code)**
  - **将 n 表示成 n 个1和最后一个0**
  - **比如：3的一元码是 1110**
  - **40的一元码**

    11111111111111111111111111111111111110
  - **70的一元码**

11111111111111111111111111111111111111111111111111111111111110

γ编码

- 将间距G表示成长度(length)和偏移(offset)两部分
- 偏移对应G的二进制编码，只不过将首部的1去掉
- 例如 13 → 1101 → 101 = 偏移
- 长度部分给出的是偏移的位数
- 比如G=13 (偏移为101), 长度部分为 3
- 长度部分采用一元编码: 1110
- G的γ编码
  - 长度部分 + 偏移部分

γ编码的例子

- **一元码：将 n 表示成 n 个1和最后一个0**
- **γ编码：长度部分 （一元码）+ 偏移部分（二进制编码）**

数　　字	一元编码	长　　度	偏　　移	γ　编　码
0	0			
1	10	0		0
2	110	10	0	10,0
3	1110	10	1	10,1
4	11110	110	00	110,00
9	1111111110	1110	001	1110,001
13		1110	101	1110,101
24		11110	1000	11110,1000
511		111111110	11111111	111111110,11111111
1025		11111111110	0000000001	11111111110 ,0000000001

练习

分别计算130的可变字节（VB）码、γ编码

解：

VB码：00000001 10000010

γ编码：11111110, 0000010

γ编码的长度

- **偏移部分是 $\lfloor \log_2 G \rfloor$ 比特位**
- **长度部分需要 $\lfloor \log_2 G \rfloor + 1$ 比特位**
- **因此，全部编码需要$2\lfloor \log_2 G \rfloor + 1$比特位**
- **γ 编码的长度均是奇数**
  - **假定间隔G的出现频率正比于$\log_2 G$—实际中并非如此)**
  - **(assuming the frequency of a gap G is proportional to $\log_2 G$– not true)**

γ编码的性质

- γ编码是**前缀无关**的
  - 一个合法的γ编码不会是任何一个其他的合法γ编码的前缀，也保证了解码的**唯一性**
  - 编码在最优编码的2或3倍之内
  - **上述结果并不依赖于间隔的分布**
- 因此，**γ编码适用于任何分布**，也就说γ编码是通用性(universal)编码
- γ编码是**无参数编码**，不需要通过拟合得到参数

γ编码的对齐问题

- 机器通常有字边界
  - 8, 16, 32 位
- 按照位进行压缩或其他处理可能会较慢

- 可变字节码通常按字边界对齐，因此可能效率更高
- 额外增加了一点点开销，但概念上简单很多

γ编码的实际应用特点

- 文档集中的词项分布
- 倒排记录表的间距分布
- 通过统计数字可以计算出采用γ编码进行倒排索引压缩的空间需求
- 960MB原始文档集的倒排索引能够压缩到大约224MB，差不多是原始文档集大小的1/4
- 实际采用γ编码对Reuters-RCV1文档集进行索引压缩时得到的索引更小，只有大约101MB，是原始文档集大小的1/10

γ编码的实际应用特点

预期大小和实际大小不一致的主要原因包括：

(i) Zipf 定律对Reuters-RCV1 文档集的词项频率实际分布的估计并不是非常准确

(ii) 间距并不满足均匀分布

γ编码的实际应用特点

Reuters RCV1语料库的统计信息

N	文档数目	800,000
L	每篇文档的词条数目	200
M	词项数目(= 词类数目)	400,000
	每个词条的字节数 (含空格和标点)	6
	每个词条的字节数 (不含空格和标点)	4.5
	每个词项的字节数	7.5
T	无位置信息索引中的倒排记录数目	100,000,000

- 对于表4-2中未经舍入处理的原始数字，采用Zipf定律进行预测，得到的索引大小为251MB
- 如果采用Zipf定律来产生词项的频率信息，并且对这些人造的词项的索引进行压缩，那么压缩后的索引大小是254MB
- 从某种程度上来说，如果关于词项分布的假设是精确的话，模型的预测结果也会准确

表5-6 Reuters RCV1索引压缩总表

数据结构	压缩后的空间大小（单位：MB）
词典，定长数组	11.2
词典，长字符串+词项指针	7.6
词典，按块存储，k=4	7.1
词典，按块存储+前端编码	5.9
文档集（文本、XML标签等）	3 600.0
文档集（文本）	960.0
词项关联矩阵	40 000.0
倒排记录表，未压缩（32位字）	400.0
倒排记录表，未压缩（20位）	250.0
倒排记录表，可变字节码	116.0
倒排记录表，γ编码	101.0

对于Reuters-RCV1 文档集，词项关联矩阵（参见图1-1）所占据的空间

$400\,000 \times 800\,000 = 40 \times 8 \times 10^9$bit，即40GB

总结

- 对于Reuters-RCV1 文档集，γ 编码技术能够取得比可变字节码更高的压缩率
  - 提高的比率大概是15%
- 但是解压的消耗会更高，主要原因是因为采用位编码时，两个编码之间的分界点可能在某个机器字当中
  - 因此在对γ编码序列解码时需要很多诸如移位或者位掩码之类的位操作
  - 在查询处理时，采用γ编码的消耗也比采用可变长字节编码要大
- 实际中到底采用哪种编码方式取决于应用的特点
  - 比如，在具体应用时，节省磁盘空间与提高查询响应时间这两者在应用中到底孰轻孰重

总结

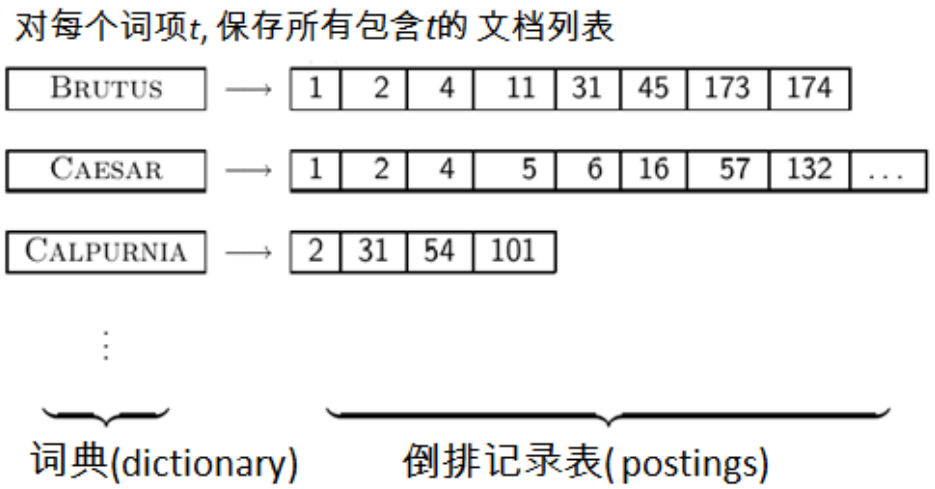
- 在表5-6中，未压缩的原始倒排索引的每条倒排记录都采用32位字存储
  - 整个索引的大小是400MB
  - 采用γ编码压缩后的索引大小为101MB
  - 采用可变长字节码压缩后的索引大小为116MB
- 两种方式下的索引压缩率都大约为25%
- 上述两种编码方式都能达到本章一开始所提到的压缩率为1/4的目标
- 通过减少索引的磁盘存储空间、增加高速缓存中的信息存放量和加快从磁盘到内存的数据传输速度，索引压缩能够大大提高索引的时空效率

总结

- 现在我们可以构建一个这样的索引
- Result
  - 空间上非常节省
  - 支持高效布尔检索
  - 大小仅为文档集中文本量的10-15%
- But
  - 没有考虑词项的出现位置和频率信息
  - 实际当中并不能达到如此高的压缩比

本讲小结

- **信息检索中进行压缩的动机**
- **词项统计量: 词项在整个文档集中如何分布？ 两个定律**
- **倒排索引中词典部分如何压缩？ 长字符串方法及改进**
- **倒排索引中倒排记录表部分如何压缩？ 可变字节码、 γ 编码**



参考资料

- 《信息检索导论》第5章
- http://ifnlp.org/ir
  - 有关字对齐二元编码的原文Anh and Moffat (2005); 及 Anh and Moffat (2006a)
  - 有关可变字节码的原文Scholer, Williams, Yiannis and Zobel (2002)
  - 更多的有关压缩 (包括位置和频率信息的压缩)的细节参考Zobel and Moffat (2006)